

# Intersection Observer API

L'Intersection Observer API est une interface JavaScript qui permet de surveiller les changements d'intersection d'un élément cible avec un élément ancêtre ou avec la fenêtre de visualisation (viewport). Cette API est particulièrement utile pour implémenter des fonctionnalités telles que le chargement paresseux (lazy loading) des images, la détection de la visibilité des éléments pour les animations, et bien plus encore.

## Création d'un Intersection Observer

Pour créer un Intersection Observer, vous devez instancier un objet `IntersectionObserver` en fournissant une fonction de rappel (callback) et une option de configuration.

```
const observer = new IntersectionObserver(fonctionRappel, options);

function fonctionRappel(elementsObservees, observer) {
  elementsObservees.forEach((element) => {
    if (element.isIntersecting) {
      console.log("L'élément est visible dans la zone d'intersection.");
    } else {
      console.log("L'élément n'est pas visible dans la zone d'intersection.");
    }
  });
}
```

- `fonctionRappel`: une fonction qui sera appelée chaque fois qu'un changement d'intersection est détecté. Cette fonction reçoit deux arguments : une liste d'objets `IntersectionObserverEntry` et l'observateur lui-même.
- `options`: un objet optionnel qui peut contenir les propriétés suivantes :
  - `root`: l'élément ancêtre utilisé comme conteneur si sera défilé. Si null, la fenêtre de visualisation est utilisée.
  - `rootMargin`: une marge autour du `root` pour ajuster la zone d'intersection.
  - `threshold`: un tableau de valeurs entre 0 et 1 qui définit les pourcentages d'intersection à observer.

## Propriétés de l'objet IntersectionObserverEntry

Chaque objet `IntersectionObserverEntry` contient plusieurs propriétés utiles, notamment :

- `boundingClientRect`: le rectangle de l'élément cible.
- `intersectionRatio`: le ratio d'intersection entre l'élément cible et le `root`.
- `isIntersecting`: un booléen indiquant si l'élément cible est actuellement intersecté avec le `root`.

## Ajout d'éléments à observer

Une fois que vous avez créé un Intersection Observer, vous pouvez ajouter des éléments à observer en utilisant la méthode `observe`. Il suffit de passer l'élément cible en argument ensuite vous pouvez modifier

l'élément avec la fonction de rappel lorsque l'intersection change.

```
const target = document.querySelector("#myElement");
observer.observe(target);
```

## Exemple complet

Voici un exemple complet qui utilise l'Intersection Observer API pour charger des images paresseusement lorsque elles entrent dans la zone de visualisation.

```
const images = document.querySelectorAll("img[data-src]");

function initialiser() {
  const options = {
    root: null,
    rootMargin: "0px",
    threshold: 0.1,
  };

  const observer = new IntersectionObserver(onChangementIntersection, options);

  images.forEach((img) => {
    observer.observe(img);
  });
}

function onChangementIntersection(entries, observer) {
  entries.forEach((entry) => {
    if (entry.isIntersecting) {
      const img = entry.target;
      img.src = img.getAttribute("data-src");
      img.removeAttribute("data-src");
      observer.unobserve(img);
    }
  });
}

window.addEventListener("load", initialiser);
```

## Références

[https://developer.mozilla.org/en-US/docs/Web/API/Intersection\\_Observer\\_API](https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API)

<https://www.youtube.com/watch?v=2lbRtjez6ag> <https://www.smashingmagazine.com/2021/07/dynamic-header-intersection-observer/>